

TD 2

Spécificités logicielles

EXERCICE #1 ► Symboles assembleurs et mémoire

Certains symboles spécifiques peuvent être définis en assembleur comme la table d'interruption. On peut aussi définir des données avec la disposition voulue. Le symbole '.' réfère l'emplacement mémoire actuellement pointé et est incrémenté automatiquement de la taille des éléments insérés.

- 1) Combien de symboles sont définis ?
- 2) Pour chaque symbole défini, donner l'adresse et la taille correspondantes.
- 3) Donner la disposition mémoire correspondante en *little-endian* et en *big-endian*.
- 4) Peut-on savoir en parcourant le symbole tab3 qu'on a atteint la fin du tableau ?
- 5) Donner le code C équivalent.

```
.data /* on suppose que . == 0x20000000 */
tabs :
    .word tab1,tab2,tab3,tab4
tab1 :
    .byte 0x00, 0x01, 0xf1, 'S'+0x20, 98
    .ascii "abc"
tab2 :
    .string "bonjour"
tab3 :
    .hword 0xf0e5,0xd42a,0x2ccb,0x12
tab4 :
    .short 234,32564,5693,-127
```

EXERCICE #2 ► Problème de compilation

Une borne de jeu propose le jeu Motus (*Wordle*) en utilisant l'ensemble des mots du dictionnaire de 6 lettres (13900 mots dans la langue française). La borne dispose d'une mémoire Flash de 128 kB et d'une mémoire SRAM de 64 kB. On propose plusieurs implémentations :

```
1 const char* dico[] = {"abaque", "abatee", ..., "zython", "zythum"};
2 const char dico[][6] = {"abaque", "abatee", ..., "zython", "zythum"};
```

- 1) Représenter l'agencement mémoire (*memory layout*) des 2 déclarations.
- 2) Calculer la taille mémoire pour chaque mot dans chacun des cas.
- 3) En déduire pourquoi la première solution ne compile pas.
- 4) Quelle serait l'avantage de la première solution, en terme de diversité de mots, si on avait suffisamment de mémoire ? Quel est l'inconvénient de la seconde lorsqu'on veut afficher un mot avec `printf` ?
- 5) Pourquoi la solution `const char** dico = {"abaque", "abatee", ..., "zython", "zythum"};` ne peut pas fonctionner ?

EXERCICE #3 ► Un peu d'acquisition et autres représentations

On souhaite mesurer des courants compris entre -20 A et $+80\text{ A}$, avec une précision de 0.1 A .

- 1) Donner le nombre total de valeurs possibles. Combien de bits sont nécessaires pour représenter toute l'étendue de la plage de mesure ?
- 2) On souhaite représenter ces courants par un nombre binaire signé qui sera mis à l'échelle pour de l'affichage ou du traitement.
 - (a) Combien de bits sont nécessaires avec cette représentation signée ?
 - (b) À quels courants correspondent les valeurs suivantes : 0 , -121 , 724 ?
 - (c) Réaliser le calcul de puissance d'un courant de 10 A traversant une résistance de $1\ \Omega$. Le résultat est-il correct ?
- 3) Finalement, on propose de représenter ces courants par un nombre binaire naturel décalé, c'est-à-dire dont la valeur '0' représente -20 A , '1' représente -19.9 A , etc.
 - (a) Combien de bits sont nécessaires en binaire décalé ?
 - (b) À quels courants correspondent les valeurs suivantes : 803 , 1000 , 42 ?
 - (c) Réaliser l'addition en binaire de 3 A et 18 A . Le résultat est-il correct ?

EXERCICE #4 ► Virgule fixe

- 1) Rappeler les bornes minimales et maximales pour un nombre virgule fixe au format $Q_{E,F}$ où E désigne le nombre de bits entiers et F le nombre de bits fractionnaire.
- 2) Donner la valeur décimale des nombres suivants au format $Q_{2,6}$: $0x87$, $0x34$, $0x58$, $0xC4$.
- 3) Réaliser les conversions suivantes au format $Q_{3,5}$: π , $-\pi$, $\sqrt{2}$, 1.5 , -1.2345
- 4) Réaliser les opérations suivantes au format $Q_{1,7}$: $\frac{-13}{128} + \frac{28}{128}$, $\frac{100}{128} \times \frac{64}{128}$, $\frac{84}{128} \times \frac{65}{128}$, $\frac{8}{128} \div \frac{96}{128}$, $\frac{43}{128} \div \frac{46}{128}$.